Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Звіт про виконання лабораторної роботи №7

«Організація шару зберігання даних (Persistence Layer)»

з курсу «Професійний Python»

Виконала:

студентка 3 курсу групи ФеП-32

Братівник Анастасія

Викладач: доц. Куньо І. М.

Львів – 2026

Мета: Ознайомитися з основними принципами організації шару зберігання даних у Python-застосунках, навчитися працювати з реляційними базами даних, мовою SQL, СУБД SQLite, стандартом Python DB-API, параметризованими запитами, CRUD-операціями, транзакціями, принципами ACID, патерном Repository, SQLAlchemy ORM та міграціями бази даних. Інтегрувати persistence layer у проєкт з попередніх робіт та перевірити його за допомогою integration tests.

Індивідуальний варіант та постановка задачі

Варіант №1 — Система обліку студентів.

Необхідно створити пов'язані таблиці groups і students (зв'язок one-to-many). Реалізувати StudentRepository та GroupRepository. Зберегти функціонал Lab_01: додавання/виведення студентів, пошук за прізвищем та групою, визначення найкращого студента та середнього балу. Додати зміну балу, переведення між групами та видалення. Реалізувати параметризований пошук через DB-API. Створити транзакційну операцію масового переведення з можливістю rollback. Додати міграцію Alembic для поля email. Написати integration tests.

Опис предметної області

Предметна область — облік студентів та їхніх академічних груп. Студент обов'язково належить до однієї групи (зв'язок "багато до одного"). Група може містити багато студентів. Кожен студент має середній бал, який можна оновлювати. Для забезпечення цілісності даних при переведенні студентів між групами використовуються транзакції.

Структура проєкту

Проєкт зберігає структуру src-layout, започатковану в Lab_01, та доповнений новими модулями для роботи з БД:

```
Brativnyk_Anastasiia_Lab07/
├── alembic.ini
```

```
├── migrations/
├── src/
│   ├── reliable_importer/    # (з Lab06)
│   └── student_manager/      # (новий Persistence Layer)
│       ├── __init__.py
│       ├── database.py       # engine та session
│       ├── models.py         # ORM-моделі SQLAlchemy
│       ├── repositories.py   # Repository Pattern
│       ├── services.py       # бізнес-логіка та транзакції
│       ├── db_api.py         # чистий DB-API (sqlite3)
│       └── main.py           # точка входу
└── tests/
    ├── conftest.py           # фікстури (in-memory БД)
    ├── integration/
    └── unit/
```

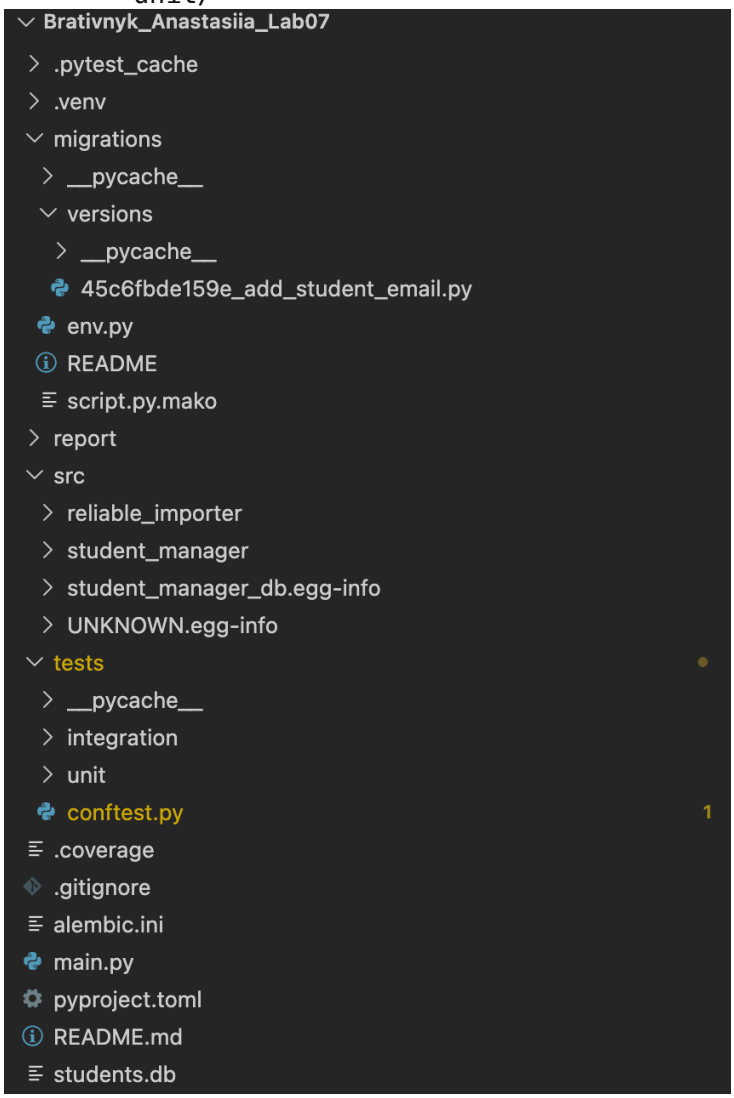


Схема бази даних

Створено дві таблиці зі зв'язком **One-to-Many**:

- **groups**: id (Primary Key), name (Unique).
- **students**: id (Primary Key), first_name, last_name, average_grade, group_id (Foreign Key -> groups.id), email.

SQL-команди створення таблиць (згенеровані SQLAlchemy)

```
CREATE TABLE groups (
    id INTEGER NOT NULL,
    name VARCHAR(50) NOT NULL,
    PRIMARY KEY (id),
    UNIQUE (name)
);

CREATE TABLE students (
    id INTEGER NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    average_grade FLOAT NOT NULL,
    group_id INTEGER NOT NULL,
    email VARCHAR(255),
    PRIMARY KEY (id),
    FOREIGN KEY(group_id) REFERENCES groups (id),
    UNIQUE (email)
);
```

Приклад Parameterized Query (DB-API)

Для захисту від SQL-ін'єкцій у модулі db_api.py використано параметризований запит із передачею параметра через ?:

```
cursor.execute(
    """
    SELECT students.first_name, students.last_name, students.average_grade
    FROM students
    JOIN groups ON students.group_id = groups.id
    WHERE groups.name = ?
    """,
    (group_name,)
)
```

ORM-моделі (models.py)

Моделі описані за допомогою сучасного синтаксису SQLAlchemy 2.0 (DeclarativeBase, Mapped, mapped_column):

```
class Group(Base):
    __tablename__ = "groups"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=T
rue)
    name: Mapped[str] = mapped_column(String(50), unique=True, nullable=False)
    students: Mapped[list["Student"]] = relationship(back_populates="group", c
ascade="all, delete-orphan")

class Student(Base):
    __tablename__ = "students"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=T
rue)
    first_name: Mapped[str] = mapped_column(String(100), nullable=False)
    last_name: Mapped[str] = mapped_column(String(100), nullable=False)
    average_grade: Mapped[float] = mapped_column(Float, nullable=False)
    group_id: Mapped[int] = mapped_column(ForeignKey("groups.id"), nullable=Fa
lse)
    email: Mapped[str | None] = mapped_column(String(255), unique=True, nullab
le=True)
    group: Mapped["Group"] = relationship(back_populates="students")
```

Реалізація Repository Pattern

Клас StudentRepository інкапсулює всі запити до БД, відокремлюючи їх від бізнес-логіки:

```
class StudentRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, student: Student) -> Student:
        self.session.add(student)
        self.session.commit()
        self.session.refresh(student)
        return student

    def get_by_id(self, student_id: int) -> Student | None:
```

```
        return self.session.get(Student, student_id)

    def update_grade(self, student_id: int, grade: float) -> Student | None:
        student = self.get_by_id(student_id)
        if student:
            student.average_grade = grade
            self.session.commit()
            self.session.refresh(student)
        return student
```

Приклад транзакції та пояснення commit/rollback

У файлі services.py реалізовано функцію масового переведення студентів. Якщо хоча б один студент не знайдений, операція повинна повністю скасуватися (принцип Atomicity):

```
def transfer_students(session: Session, student_ids: list[int], new_group_id:
int) -> None:
    try:
        for student_id in student_ids:
            student = session.get(Student, student_id)
            if student is None:
                raise ValueError(f"Student {student_id} not found")
            student.group_id = new_group_id
        session.commit()  # Зберігаємо зміни, якщо все ОК
    except Exception:
        session.rollback() # Відкочуємо всі зміни при помилці
        raise
```

- commit() — фіксує транзакцію в базі даних, роблячи зміни постійними (Durability).
- rollback() — скасовує всі незафіксовані зміни поточної транзакції, повертаючи БД до попереднього стану (Atomicity).

Пояснення ACID:

- **Atomicity**: Транзакція виконується повністю або не виконується зовсім (як у нашому transfer_students).
- **Consistency**: БД переходить з одного коректного стану в інший (дотримуються зовнішні ключі).

- **Isolation**: Одночасні транзакції не впливають один на одного.
- **Durability**: Після commit() дані зберігаються навіть при збоях.

Міграція (Alembic)

За допомогою Alembic було згенеровано міграцію для додавання поля email:

```
alembic revision --autogenerate -m "add student email"
alembic upgrade head
```

Згенерований код у migrations/versions/:

```
def upgrade() -> None:
    op.add_column('students', sa.Column('email', sa.String(length=255), nullable=True))
    op.create_unique_constraint(None, 'students', ['email'])
```

```
● (.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab07 % alembic downgrade base

  INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
  INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
  INFO  [alembic.runtime.migration] Running downgrade 45c6fbde159e -> , add student email
● (.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab07 % alembic upgrade head

  INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
  INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
  INFO  [alembic.runtime.migration] Running upgrade  -> 45c6fbde159e, add student email
● (.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab07 % alembic current

  INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
  INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
  45c6fbde159e (head)
○ (.venv) anastasiabrativnik@MacBook-Air-Anastasia-2 Brativnyk_Anastasiia_Lab07 %
```

Integration tests

Тести написані з використанням ізольованої бази даних у пам'яті (sqlite:///:memory:), що створюється у фікстурі db_session (у conftest.py).

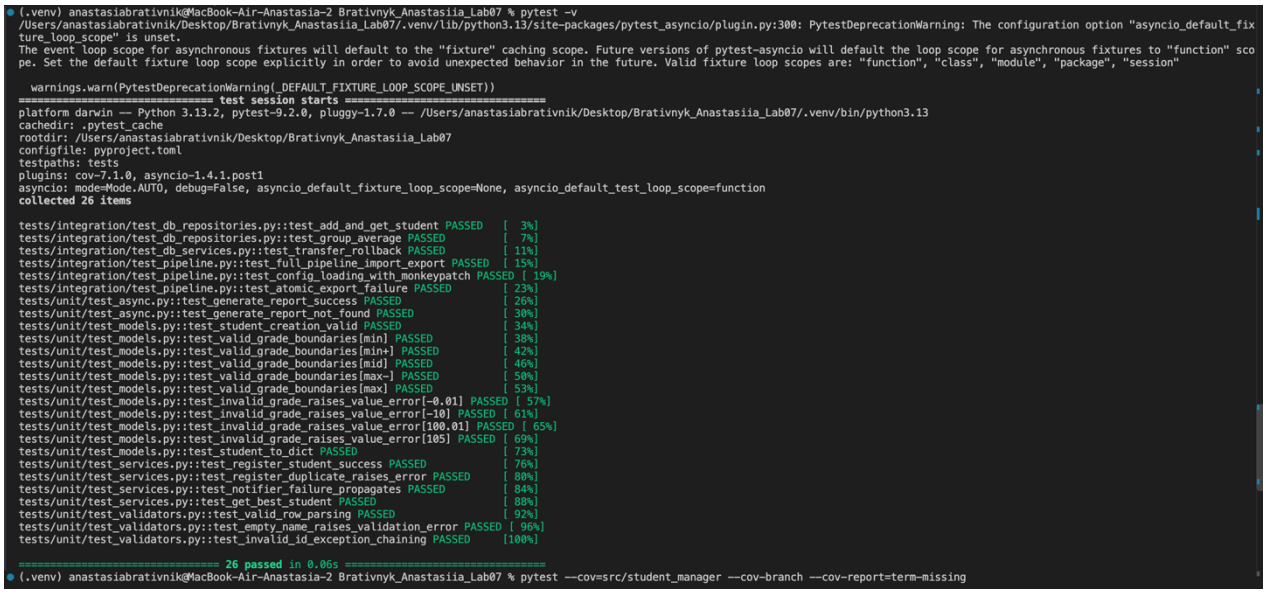
```
@pytest.mark.integration
def test_transfer_rollback(db_session):
    g1 = GroupRepository(db_session).add("PI-21")
    g2 = GroupRepository(db_session).add("PI-22")
    repo = StudentRepository(db_session)
    student = repo.add(Student(first_name="Anna", last_name="Koval", average_grade=91.5, group_id=g1.id))

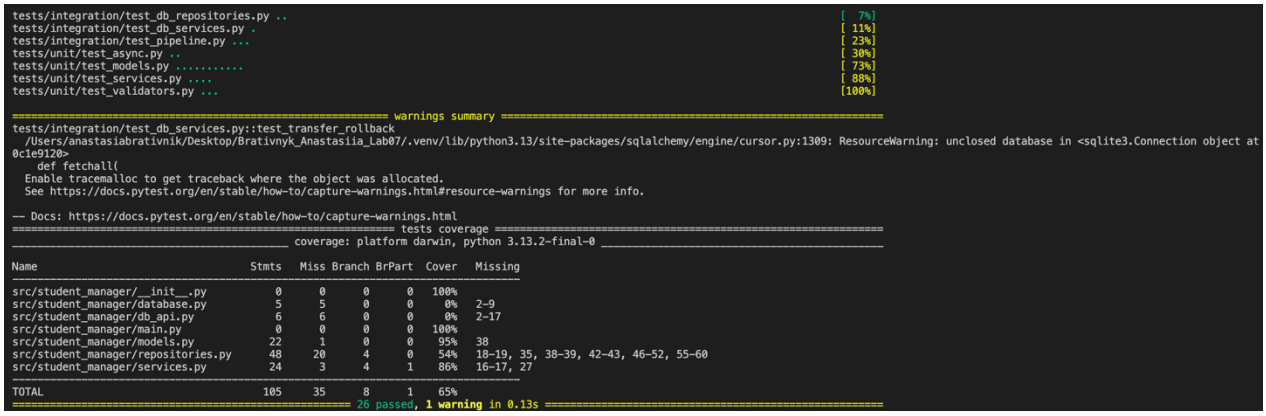
    with pytest.raises(ValueError):
        # Намагаємось перевести існуючого та неіснуючого (999999) студента
        transfer_students(db_session, [student.id, 999999], g2.id)

    db_session.refresh(student)
```

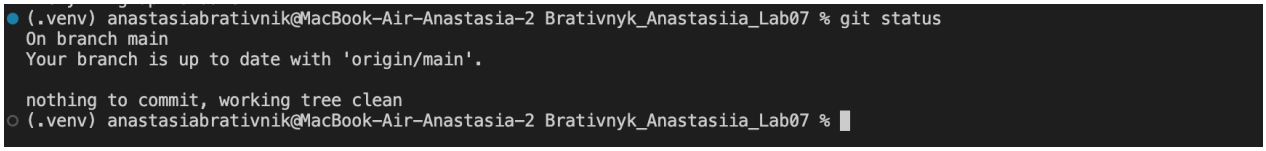
```
    assert student.group_id == g1.id  # Перевіряємо, що rollback спрацював
```

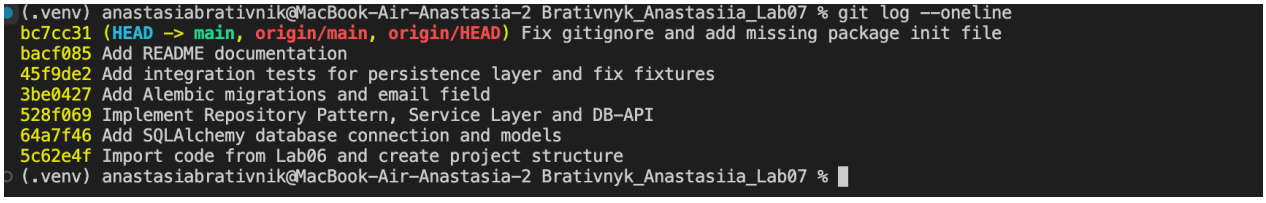
Результати тестування

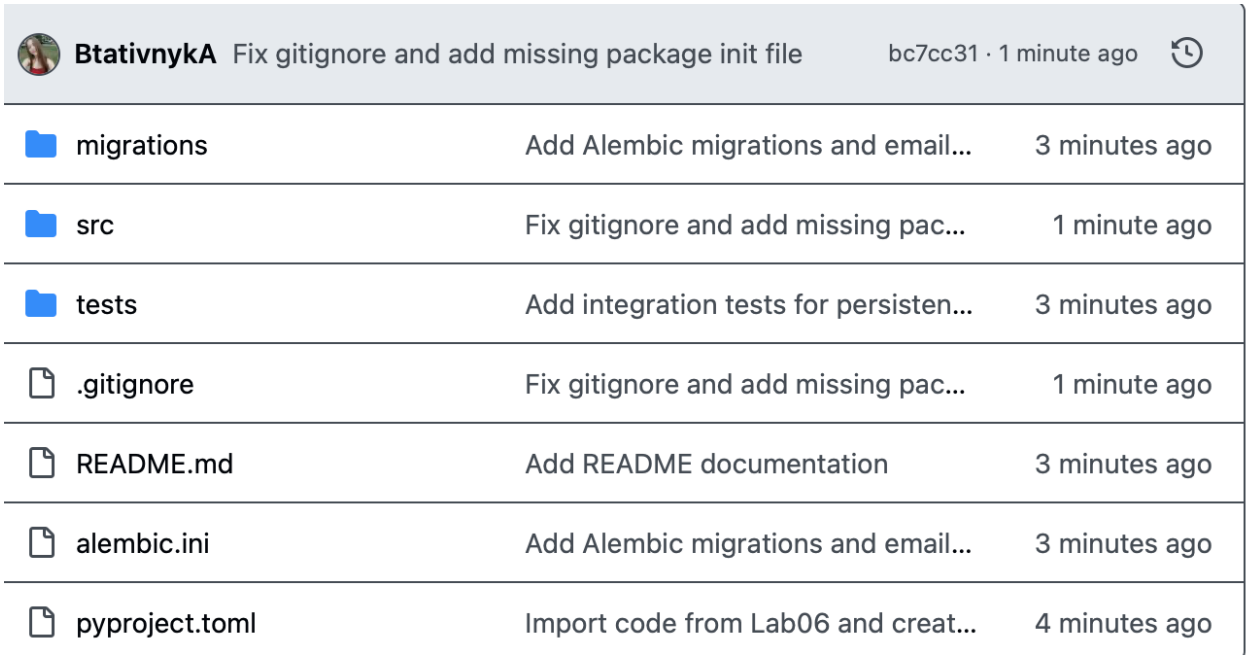




Робота з Git







Висновок:

У ході лабораторної роботи №7 реалізовано повноцінний шар збереження даних (Persistence Layer). Створено базу даних SQLite з пов'язаними таблицями groups та students (зв'язок One-to-Many). За допомогою SQLAlchemy ORM створено моделі даних. Реалізовано патерн Repository для абстрагування запитів до БД. Продемонстровано роботу з чистим DB-API (sqlite3) для параметризованих запитів. Реалізовано транзакційну логіку переведення студентів із використанням commit та rollback, що забезпечує властивість Atomicity. За допомогою Alembic успішно застосовано міграцію для додавання поля email. Робота перевірена integration-тестами з використанням in-memory SQLite, які успішно тестують CRUD-операції, агрегацію та відкат транзакцій. Усі 26 тестів пройдено успішно.